

Aula 09

*DataPrev (Perfil 3: Desenvolvimento de
Software) Banco de Dados - 2026
(Pós-Edital)*

Autor:

Lucas Fargoni Di Ianni

09 de Julho de 2026

Índice

1) NoSQL (conceitos) - Teoria	3
2) SGBDs NoSQL - Teoria	20
3) NoSQL - Questões Comentadas	43
4) NoSQL - Lista de Questões	52

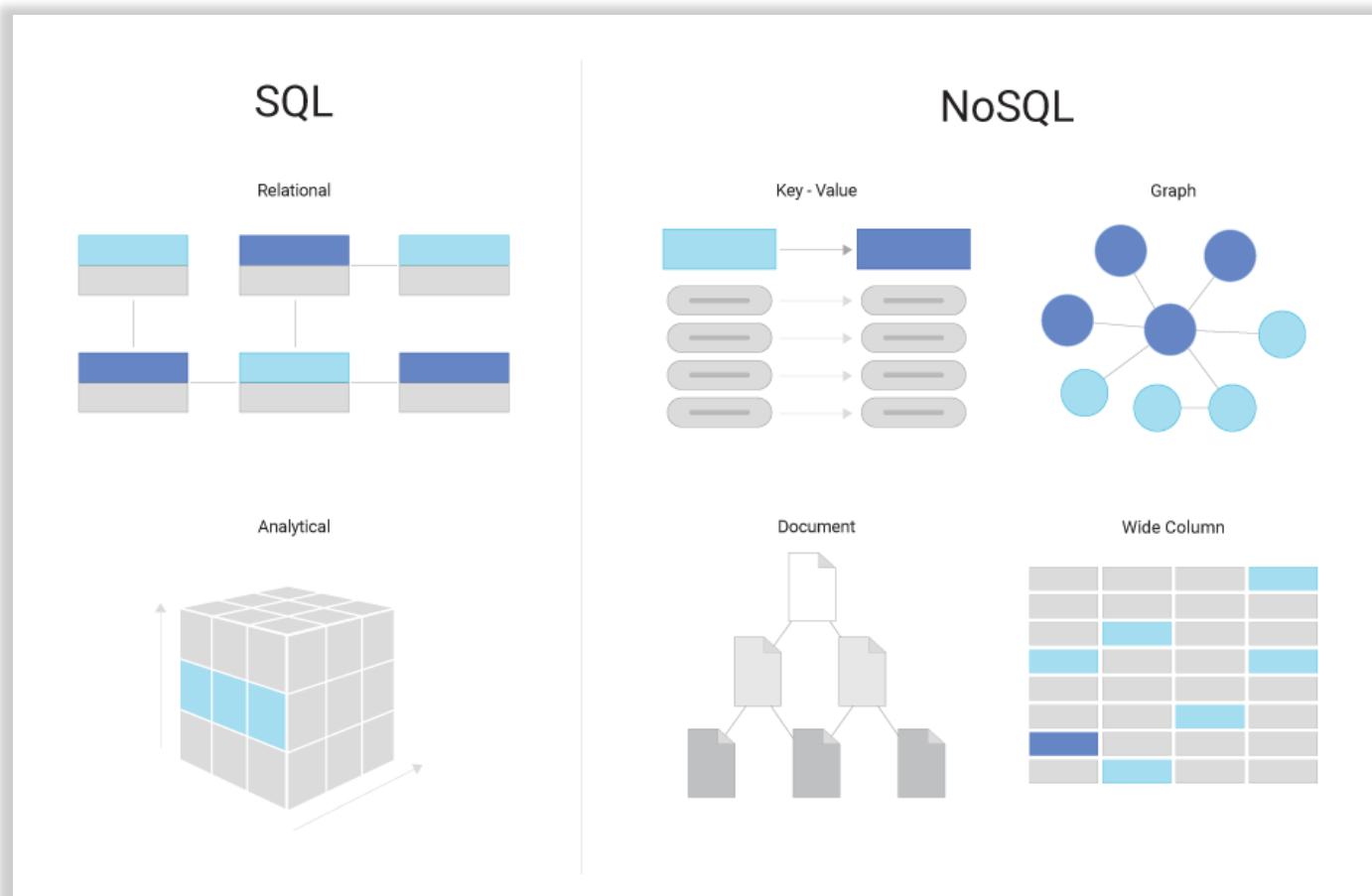
NoSQL

Conceitos Gerais

Até agora, todos os modelos de bancos de dados que vimos são estruturados em tabelas, sejam elas bidimensionais, como o modelo relacional, ou multidimensionais, como o modelo utilizado pelos Data Warehouses. Agora, iremos ver modelos de bancos de dados **desestruturados**.

Cuidado! Os **modelos** que são desestruturados, já que já vimos a aplicação de dados não estruturados anteriormente.

Esses bancos de dados são chamados de **NoSQL**. A sigla não tem um significado definido, dividindo a doutrina em duas definições majoritária: Non SQL (Não-SQL) ou Not Only SQL (Não apenas SQL). Um cuidado com esse tópico - em 2023, a banca FGV cobrou questões de SQL com base na presença de NoSQL no edital. Muitas judicializações foram feitas, então não deve ser algo que irá se repetir - mas fique de olho.



Os bancos de dados NoSQL não são 1 tipo de bancos de dado, e sim um conjunto de bancos de dados com **modelos semi-definidos**. Mas, antes de estudarmos os modelos, precisamos entender como esses bancos de dados se estruturam.

Computação Distribuída

Os bancos de dado NoSQL consomem, primariamente, dados não estruturados e semiestruturados. Eles são os principais bancos de dados a lidar com os dados massivos produzidos hoje e, portanto, os **principais bancos de dados a lidar com os Big Data**.

Graças a isso, os modelos NoSQL andam lado a lado com a **computação distribuída**. A computação distribuída é um modelo de processamento de informações no qual as tarefas são divididas em várias partes menores e processadas simultaneamente em **múltiplos computadores ou servidores**.

Este modelo permite que os sistemas de computação compartilhem recursos e comuniquem-se entre si para atingir um objetivo comum. A computação distribuída é frequentemente usada para **melhorar a eficiência e o desempenho**, pois permite que grandes volumes de dados sejam processados mais rapidamente.

As redes de computação distribuídas são distribuídas em **nós**, ou *nodes*, onde cada um desses nós recebe uma função específica. Essa função varia de acordo com o tipo de arquitetura do conjunto. Temos dois tipos de destaque para o NoSQL.

Cliente-Servidor

Na arquitetura cliente-servidor, os nós são divididos em dois tipos: **clientes** e **servidores**. Os **nós cliente** são os nós que irão realizar o processamento, chamados de **nós trabalhadores**, ou **working nodes**. Já os nós que “mandam” nos outros, são os **nós servidor**. Eles são responsáveis por distribuir requisições e gerenciar os demais nós, garantindo que estão funcionando corretamente.

Esses nós costumam se comunicar num esquema de **requisição e resposta** ou, como costuma aparecer, pelo termo em inglês **request/response** (req/res). Nesse sistema de comunicação, os **clientes mandam pedidos ao servidor**, que os responde com as *responses*. Esses pedidos podem ser alocação de recursos, ser “liberado” para executar determinada tarefa, entre outros.



(CEBRASPE/TELEBRAS/2022) Julgue o próximo item, relativos a UML 2, Scrum 2020, arquitetura cliente-servidor e alta disponibilidade.

Por se tratar de uma arquitetura distribuída, o modelo cliente-servidor pressupõe facilidades para atualizar os servidores de forma transparente, sem que isso afete outras partes do sistema.

Comentários:

Essa é uma das grandes vantagens do modelo cliente-servidor. Podemos realizar implementações e manutenções em uma das partes, sem afetar as outras partes envolvidas. (Gabarito: Certo)

Arquitetura ponto a ponto (P2P)

A arquitetura **ponto a ponto**, ou **peer-to-peer** (P2P) é um sistema único, já que ele opera **sem um controle centralizado**. Podemos dizer, portanto, que a arquitetura P2P se trata de um sistema distribuído descentralizado. Aqui, cada nó possui uma conexão direta com outros nós (chamados de **nós vizinhos**), funcionando **tanto como cliente, quanto como servidor**.

Um detalhe é que podemos ter conexões entre todos os nós de uma rede - ou seja, se tivermos 25 nós em uma rede, cada unidade computacional se conectará aos outros 24 nós. Isso caracteriza o que chamamos de **mesh network**, ou **rede em malha**. Isso garante que as redes P2P possuam o atributo de **auto-organização**, permitindo a adição e remoção livre de nós.

(CEBRASPE/POLC AL/2023) A respeito de redes de comunicação de dados e assuntos correlatos, julgue o item que se segue.

Em uma rede ponto a ponto (peer to peer) de computadores, que não depende de servidores interconectados, cada ponto torna-se tanto um cliente quanto um servidor, possibilitando a troca de informações entre si ou até mesmo compartilhando periféricos conectados à rede.

Comentários:

Perfeito! Na P2P cada nó funciona tanto como servidor, quanto como cliente. Dessa forma, os nós podem compartilhar recursos, equipamentos, entre outros. (Gabarito: Certo)

Bom, mas por que eu passei tudo isso a vocês se é um assunto atinente mais à Engenharia de Softwares do que aos bancos de dados?

Na computação, tudo anda "lado a lado", a escolha pelo tipo de arquitetura distribuída impacta diretamente no funcionamento e na própria arquitetura dos bancos de dados NoSQL - resultando nos três princípios estruturantes principais de sua arquitetura. Vamos ver esses princípios.

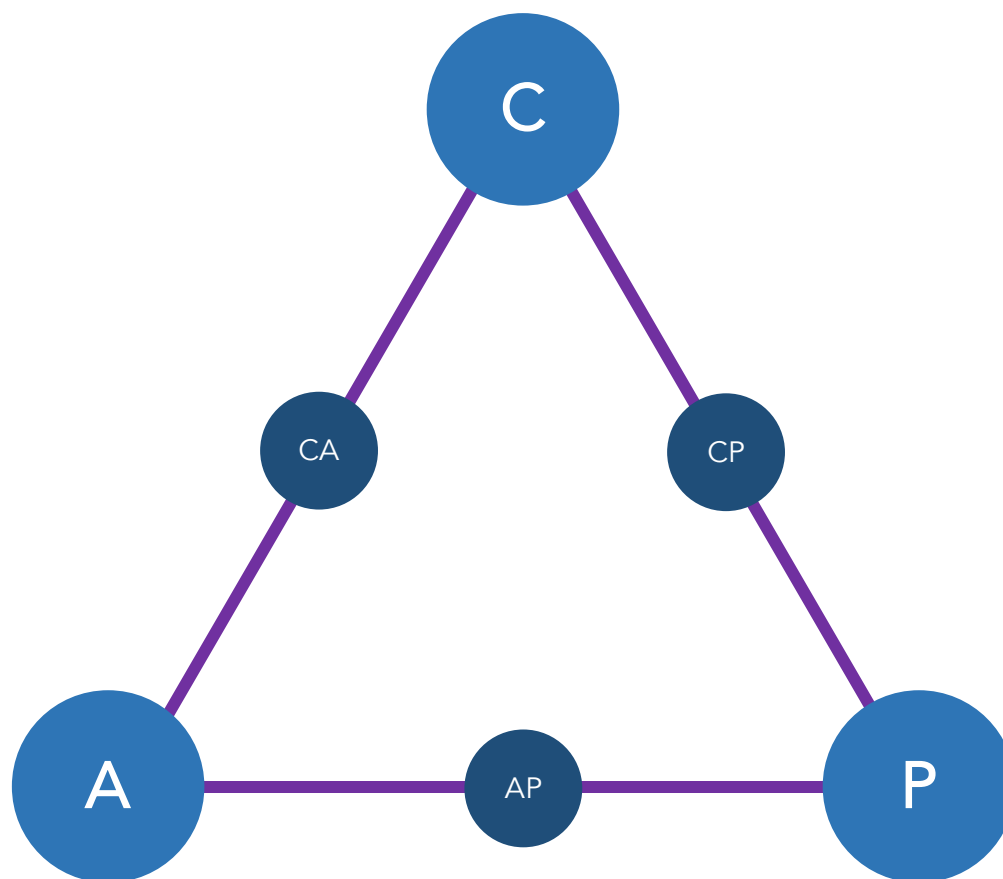
Teorema CAP

O **teorema CAP** é uma teoria da ciência computacional, também nomeada de **Teorema de Brewer**, e trata da armazenagem distribuída de dados. Já que os bancos de dados NoSQL costumam usar a computação distribuída, temos esses dados distribuídos em diversos nós - e as vezes, o dado necessário a ser acessado se encontra em um nó completamente diferente.

Esse teorema define que um banco de dados distribuído pode garantir, simultaneamente, **apenas duas, das três características** abaixo:

- **Consistência/Consistency (C):** Refere-se à propriedade de um banco de dados que garante que todas as réplicas de dados em um sistema estejam sempre atualizadas e sincronizadas, o que significa que, para qualquer leitura de dados, o mesmo valor é sempre retornado em todas as réplicas.
- **Disponibilidade/Availability (A):** Refere-se à propriedade de um banco de dados que garante que os dados estejam sempre disponíveis para leitura e gravação, mesmo que ocorra uma falha em parte do sistema.
- **Tolerância a partições/Partition Tolerance (P):** Refere-se à propriedade de um banco de dados que garante que o sistema continue a funcionar mesmo quando ocorre uma falha na rede ou em parte do sistema, permitindo que diferentes partes do sistema possam trabalhar de forma independente e sem afetar o restante

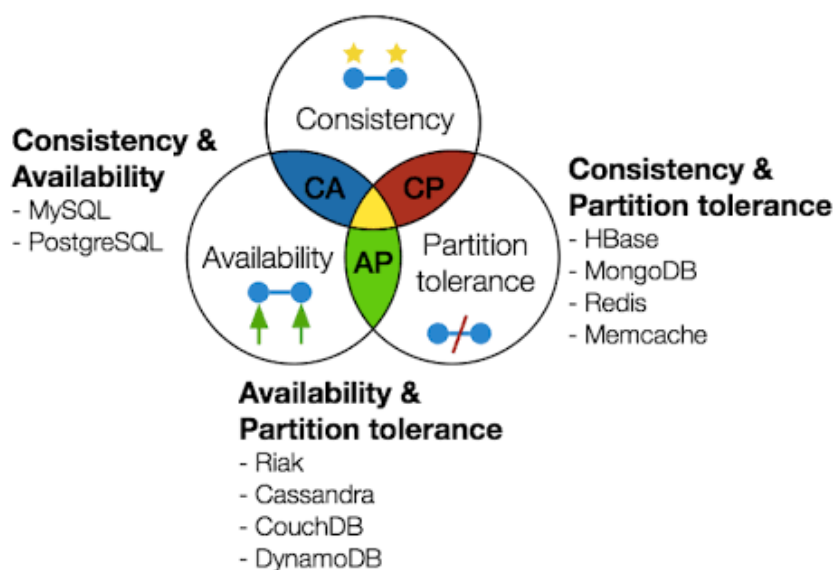
Consistência



Disponibilidade

Particionamento

Perceba que, entre os princípios, temos um campo onde é oferecido ambos: **CA** temos consistência e disponibilidade, mas não podemos particionar os dados; **CP** temos consistência e tolerância ao particionamento, mas não temos disponibilidade; **AP** temos disponibilidade e tolerância ao particionamento, mas não teremos consistência.



Cada SGBD que aborda os bancos de dado NoSQL, lida com um conjunto de princípios diferente. Por exemplo, MongoDB e Redis, dois dos principais SGBDs NoSQL, garantem a consistência e o

particionamento, sem entregar disponibilidade (portanto, CP). Veremos mais sobre os SGBDs na próxima aula.

(FGV/RFB/2023) No âmbito de bancos dados, especialmente NoSQL, o Teorema CAP sustenta que há três requerimentos básicos que existem numa relação especial quando se projeta uma aplicação com uma arquitetura distribuída. Basicamente, respalda a tese de que não se pode obter os três simultaneamente.

Esses três requerimentos são

- a) Consistency, Atomic transactions e Partition.
- b) Concurrency Partition, Availability e Tolerance.
- c) Consistency, Availability e Partition Tolerance.
- d) Consistency, Atomic transactions e Persistency.
- e) Caching, Availability e Persistency.

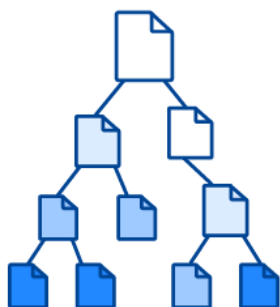
Comentários:

Como vimos, CAP é um acrônimo para "Consistency, Availability, Partition Tolerance". (Gabarito: Letra C)

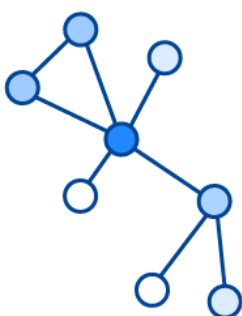
1Tipos de NoSQL

O NoSQL é um gênero, que abriga diversos tipos de bancos de dados. Para fins de prova, temos quatro tipos associados ao NoSQL que você precisa saber: **key-value**, **document oriented**, **graph oriented** e **column oriented** (ou **wide-column**).

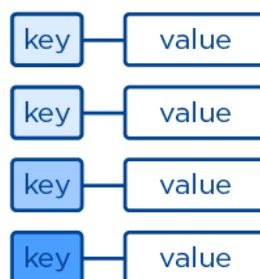
Document



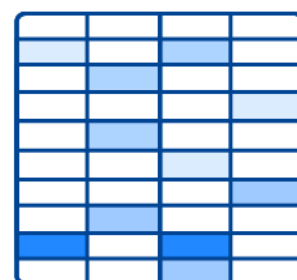
Graph



Key-Value



Wide-column



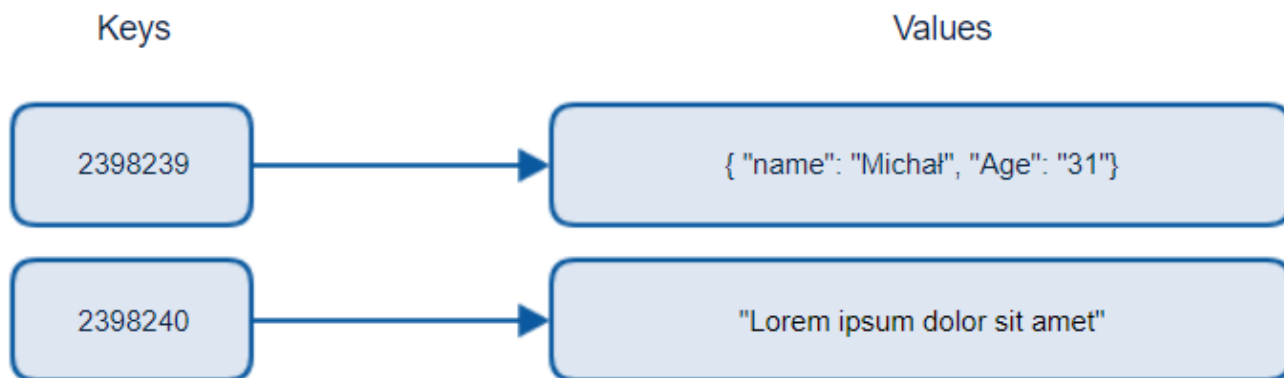
Esse assunto é extremamente importante, então preste bastante atenção.

PRESTE MAIS ATENÇÃO!



Key-Value

O banco de dados **Key-Value**, ou **chave-valor**, é um tipo de banco de dados NoSQL que armazena dados como um conjunto de pares chave-valor, onde a chave é um identificador único que faz referência a um valor associado. É um modelo de dados simples, com alta escalabilidade horizontal e desempenho rápido, o que o torna adequado para aplicativos com grandes quantidades de dados e alta taxa de leitura/gravação.



Cada chave funciona como um **identificador único** para os valores, funcionando quase como uma chave primária e um índice, ao mesmo tempo. Devido a essa característica e, por muitas vezes, usar a memória *cache* dos computadores, o Key-Value é um excelente banco de dados em termos de desempenho de consulta.

Nos valores, podemos ter literalmente qualquer coisa associada - uma imagem, um texto, um vídeo, uma música. Até mesmo diferentes tipos de valores associados a uma mesma chave, sem qualquer limitação do que seja - quem faz essa limitação é justamente quem opera o banco de dados. É interessante que essa limitação seja feita, para não termos um banco de dados completamente desordenado.

Um ponto negativo é que o chave-valor oferece **apenas operações de leituras simples**, sem suporte a consultas complexas ou relacionamentos entre os objetos. Devido a esse ponto, é indicado que o chave-valor seja implementado em bancos de dados com estruturas flexíveis e mais simples.

Para lidar com bancos de dado desse tipo, os principais SGBDs são o **Redis**, que opera com as chaves a partir de hashes criptográficos, e o **Riak**. Lembrando que ambos serão vistos na próxima aula.

QUESTÃO DE PROVA



(FGV/RFB/2023) Bancos de dados NoSQL são usualmente divididos em categorias de store.

Assinale a opção que apresenta o tipo de store que privilegia velocidade, capacidade de leitura e escrita e estruturas de dados flexíveis, sem a necessidade de esquemas estabelecidos previamente.

- a) Document.
- b) Key Value.
- c) Wide-Column.
- d) Graph.
- e) Full Text.

Comentários:

Apesar de não termos vistos os outros tipos de NoSQL ainda, quero que você guarde que, quando falamos de velocidade, capacidade de leitura e flexibilidade, estamos tratando do Key Value. (Gabarito: Letra B)

Document Oriented

Os bancos de dado NoSQL **orientados a documento** são bancos de dados que armazenam os dados em - pasmem - documentos. Esses documentos são conjuntos de caracteres que podem ser estruturados de diversas formas. Os formatos de documento que ganham mais destaque são o JSON e o XML. Alternativamente, temos o formato BSON, uma conversão binária dos arquivos em JSON.

Como é um conhecimento importante, vou "alugar" um pouco do seu tempo para explicar o que é cada um desses tipos de documentos.

- **JSON** é um formato de documento estruturados em pares de chaves e valor. Esse documento é caracterizado pela alta orientação a representação de objetos. Sua

```
{  
  "nome": "Maria",  
  "idade": 30,  
  "cidade": "São  
Paulo"  
}
```

- **BSON** é uma extensão do formato JSON que codifica os dados em binário, tornando-o mais eficiente e compacto para armazenamento e transferência de dados. Por ser binário, sua sintaxe é bem ilegível para leigos

```
\x16\x00\x00\x00\x02nome\x00\x06\x00\x00\x00Maria\x00\x02  
idade\x00\x1E\x00\x00\x00\x02cidade\x00\x0B\x00\x00\x00Sã  
o Paulo\x00\x00
```

- **XML** é um formato de dados que usa tags para marcar elementos de dados e descrever a estrutura dos dados. É frequentemente usado em aplicativos empresariais e para intercâmbio de dados entre diferentes sistemas.

```
< Pessoa>  
  < nome> Maria</ nome>  
  < idade> 30</ idade>  
  < cidade> São Paulo</ cidade>  
</ Pessoa>
```

Os bancos de dados orientados a documentos são altamente escaláveis e podem suportar grandes volumes de dados com facilidade. Eles são particularmente úteis para aplicativos com grande quantidade de **dados não estruturados ou semi-estruturados**, como logs de servidor, mensagens de mídias sociais, e-mails, entre outros.

Os bancos de dados orientados a documentos geralmente possuem uma API de consulta flexível, que permite a execução de consultas de forma complexa. Por exemplo, em um banco de dados de produtos de comércio eletrônico, podemos executar consultas para encontrar todos os produtos que correspondem a determinada categoria ou que estão disponíveis em uma determinada região geográfica.

API é uma interface entre diferentes softwares que permite que eles interajam e compartilhem dados de forma estruturada. As APIs podem ser usadas para acessar serviços web, integrar sistemas, desenvolver aplicativos móveis e muito mais.

O principal SGBD orientado a documentos é o **MongoDB**. Nele, cada dado é apresentado como um documento isoladamente. Esse dado pode ser complexo e composto de diversos valores - mas um documento é somente um dado.

(CEBRASPE/MP TCE-SC/2023) Julgue o próximo item, relativos a otimização de consultas, ETL, modelo entidade-relacionamento e bancos de dados NoSQL.

Com exceção do XML, os documentos que podem ser armazenados e recuperados em um banco de dados NoSQL de documentos são JSON e BSON.

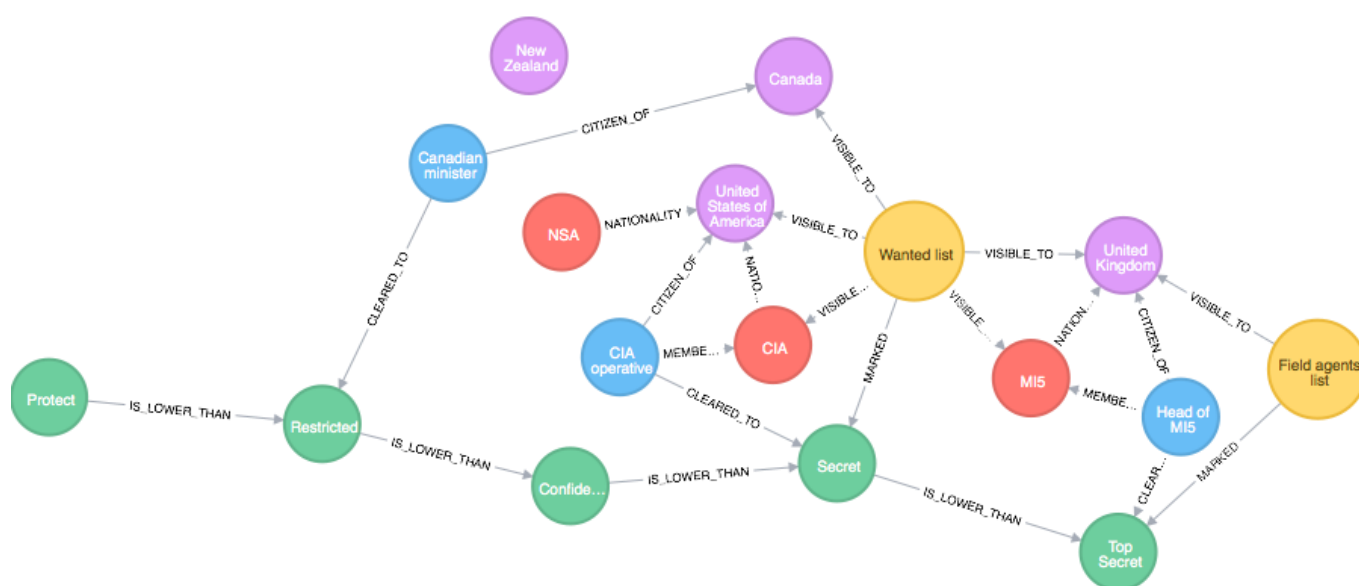
Comentários:

O NoSQL trabalha **inclusive** com o XML. Nesse sentido, incorreta a afirmativa. (Gabarito: Errado)

Graph NoSQL

Um **Graph NoSQL**, ou **modelo orientado a grafos**, armazena os dados numa estrutura de nós e relacionamentos, chamada de **grafo**. Cuidado, não são gráficos, são grafos! As bancas mesmo confundem em alguns casos, mas saiba que gráficos é uma tradução equivocada.

Aqui, os valores, os conjuntos de dados, são armazenados em **vértices**, ou **nós**, e as **arestas** que conectam esses nós exprimem seus **relacionamentos**. Veja como se parece a estrutura de um desses bancos de dados:



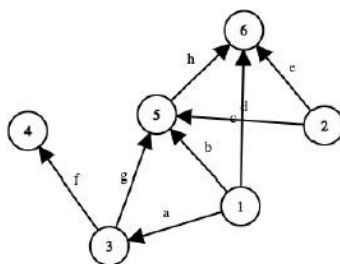
Uma das principais características de um banco de dados orientado a grafo é que **ele oferece consultas eficientes para relações complexas entre os dados**, como recomendações de amigos em redes sociais ou caminhos mais curtos em uma rede de rotas. Isso é possível porque os bancos de dados orientados a grafo permitem que as consultas sejam realizadas diretamente na estrutura do grafo, em vez de serem executadas por meio de operações complexas de junção ou agregação.

Os principais SGBDs de grafo são o **Neo4j** e o **Apache Giraph**.

QUESTÃO DE PROVA



(FGV/TJ RN/2023) Observe a estrutura do dataset ilustrado a seguir.



Para armazenar o dataset ilustrado, é necessário utilizar o banco de dados NoSQL:

- a) grafo;
- b) colunar;
- c) chave-valor;
- d) criptografado;
- e) orientado a documento.

Comentários:

Um *dataset* é um conjunto de dados e relacionamentos. Nesse caso, o *dataset* está sendo represando em um grafo - sendo, portanto, necessário um NoSQL orientado a grafos para armazená-lo. (Gabarito: Letra A)

Wide Column

Os NoSQL **orientados a colunas**, ou **wide-column**, é um tipo de banco de dados "semi tabular". Nele, cada linha de uma tabela representa um objeto - e esse objeto possui diversos dados relacionados a ele. Esses dados são estruturados em colunas. Veja:

Britney	emailAddress	gender
	brit@example.com	female
	1465676432	1465676432

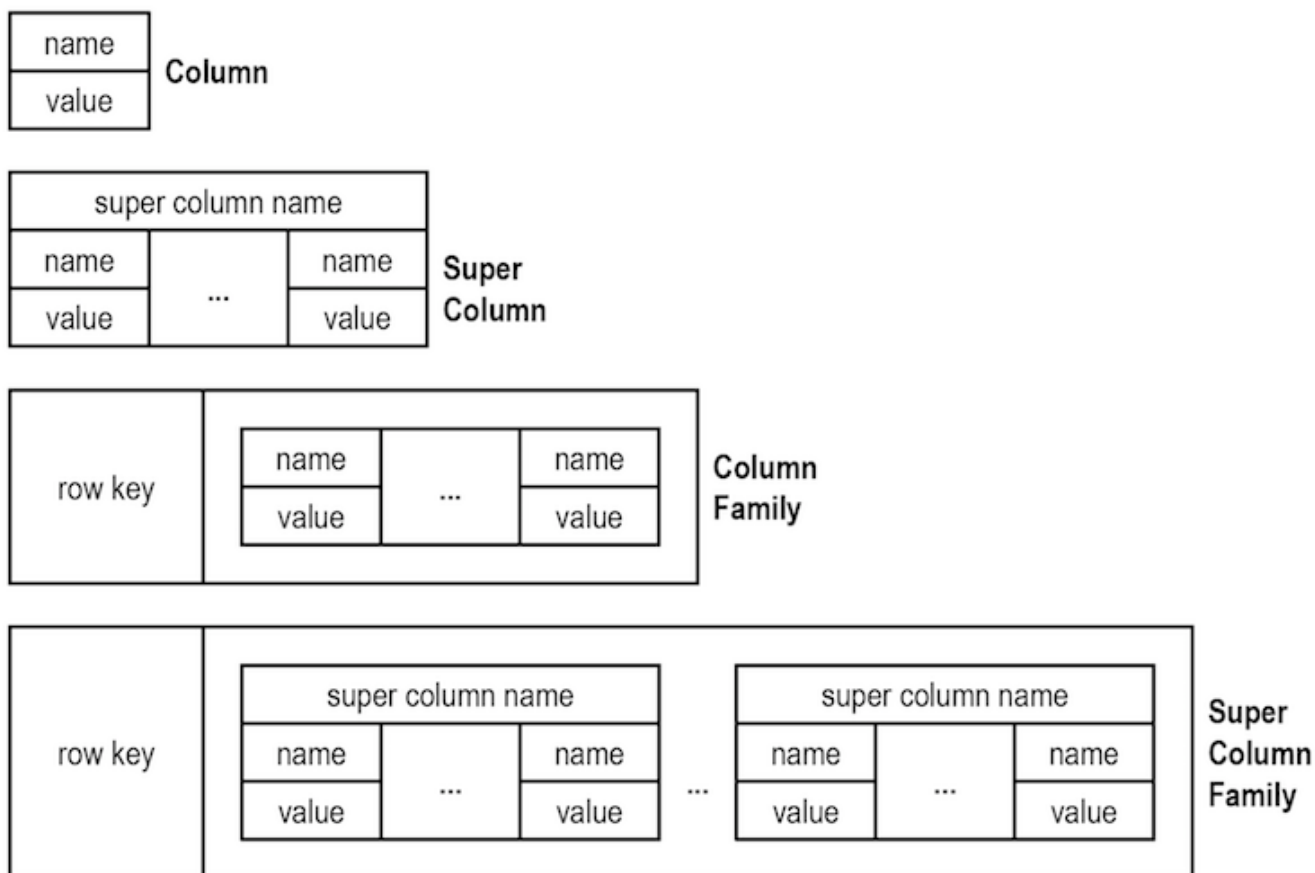
Tori	emailAddress	country	hairColor
	tori@example.com	Sweden	Blue
	1435636158	1435636158	1465633654

Cada linha será uma coletânea de colunas, que expressam os valores do objeto. Os objetos podem, ou não, ser contemplados por todas as colunas. No exemplo acima, temos duas linhas - Britney e Tori. Cada linha possui colunas, como endereço de e-mail, gênero e país, com valores associados.

Então, diferentemente das tabelas dos bancos de dados relacionais, aqui cada linha pode ter uma estrutura de colunas diferentes - e, em cada linha, uma mesma coluna pode receber diferentes nomes.

Um processo importante nos Wide Column é a **indexação**. Como podemos ter diversas versões de um mesmo dado, de uma mesma linha, nós usamos um conjunto de **três valores** para a indexação do dado: a **coluna**, a **linha** e um **carimbo de tempo** (*timestamp*), que irá referenciar a versão do dado abordada.

Para melhor entender a estrutura, preciso que você entenda o conceito de **colunas**, **super colunas** e **famílias de colunas**. As **super colunas**, ou **wide columns**, são **colunas compostas de outras colunas**. Dentro de uma super coluna, podemos ter diversas outras colunas, com seus respectivos valores. A partir do momento que agrupamos colunas e super colunas relacionadas a um único objeto, temos o que é chamado de **famílias de colunas**. É justamente o que vimos no exemplo acima - Tori e Britney é uma família de colunas, referentes a um objeto cada.



Um dos principais benefícios de um banco de dados orientado a colunas é **a capacidade de lidar com grandes volumes de dados e consultas complexas de forma eficiente**. Ele também oferece maior flexibilidade na adição e remoção de colunas, em comparação com bancos de dados relacionais.

Os principais SGBDs de dados colunares são o **Cassandra**, o **Amazon Redshift** e o **Microsoft Azure Cosmos DB**.

ESTA É DIFÍCIL!



(CEBRASPE/EMPRESA/2023 - Adaptada) Julgue o item abaixo acerca dos conhecimentos sobre os bancos de dados NoSQL orientados a colunas.

Nesse banco de dados, os dados são indexados por uma tripla (linha, coluna e timestamp).

Comentários:

Perfeito. Para indexar os dados nos *wide column*, usamos três valores: a linha, a coluna e um carimbo de tempo, ou *timestamp*. (Gabarito: Certo)

SGBDs NoSQL

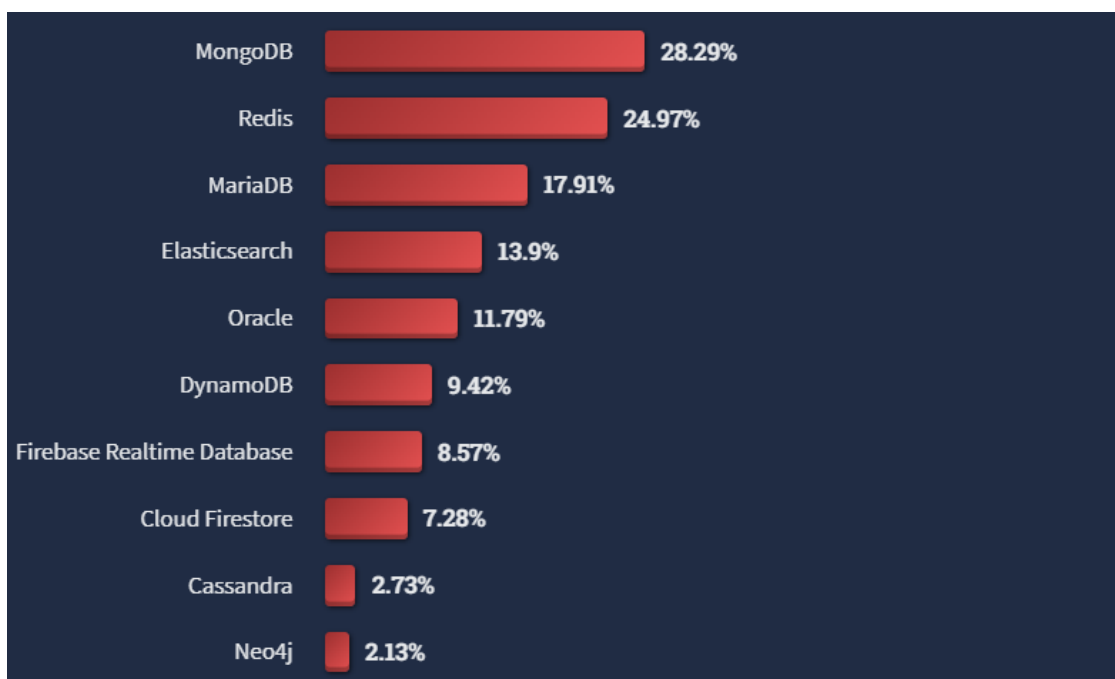
Conceitos Gerais

Vamos ver os principais **SGBDs de NoSQL**.

Começo com uma má notícia: vejam que eu falei os **principais**. Isso pois temos uma **infinidade** de SGBDs NoSQL e, usualmente, as bancas simplesmente colocam “SGBD NoSQL” no edital, sem especificar. Mas, por sorte, as cobranças tendem a rodar em torno de poucos programas.



Vamos ter uma visão mais aprofundada em três SGBDs: **mongoDB**, **Cassandra** e **Redis**. Também vamos dar uma passada por cima do **Dynamo**, **CouchDB**, **Apache HBase** e **Neo4j**. A cobrança costuma se orientar pelos três “grandes”. E faz sentido, são alguns dos mais populares:



MongoDB

Conceitos Gerais

MongoDB é o SGBD NoSQL **mais utilizado** no mundo. Só perde para bancos de dados relacionais, como MySQL e PostgreSQL.

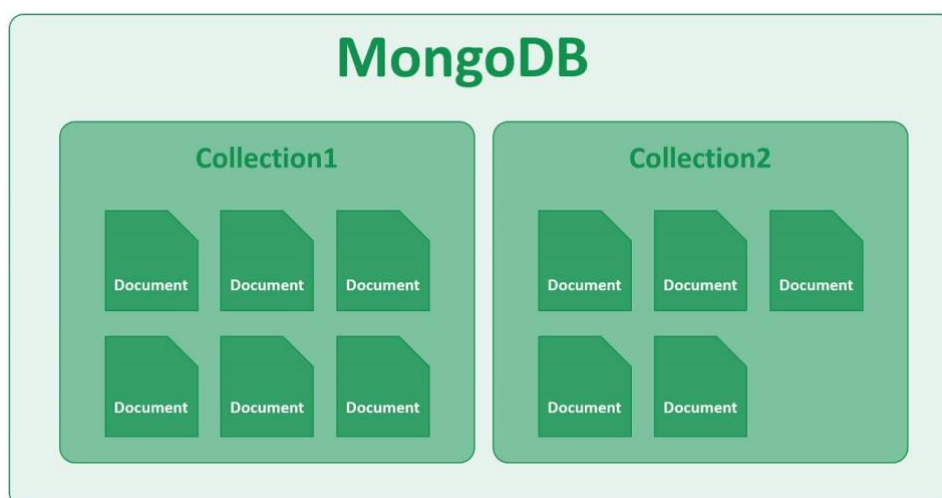
Ele é um banco de dados dedicado a tratar de documentos, portanto, de NoSQLs **document oriented**. A sua característica identificadora é a utilização de arquivos no formato **BSON** (Binary JSON), que é uma conversão binária de arquivos JSON – que, por sua vez, são arquivos que representam objetos em pares campos e valores.

```
{  
  name: "sue",  
  age: 26,  
  status: "A",  
  groups: [ "news", "sports" ]  
}
```

← field: value
← field: value
← field: value
← field: value

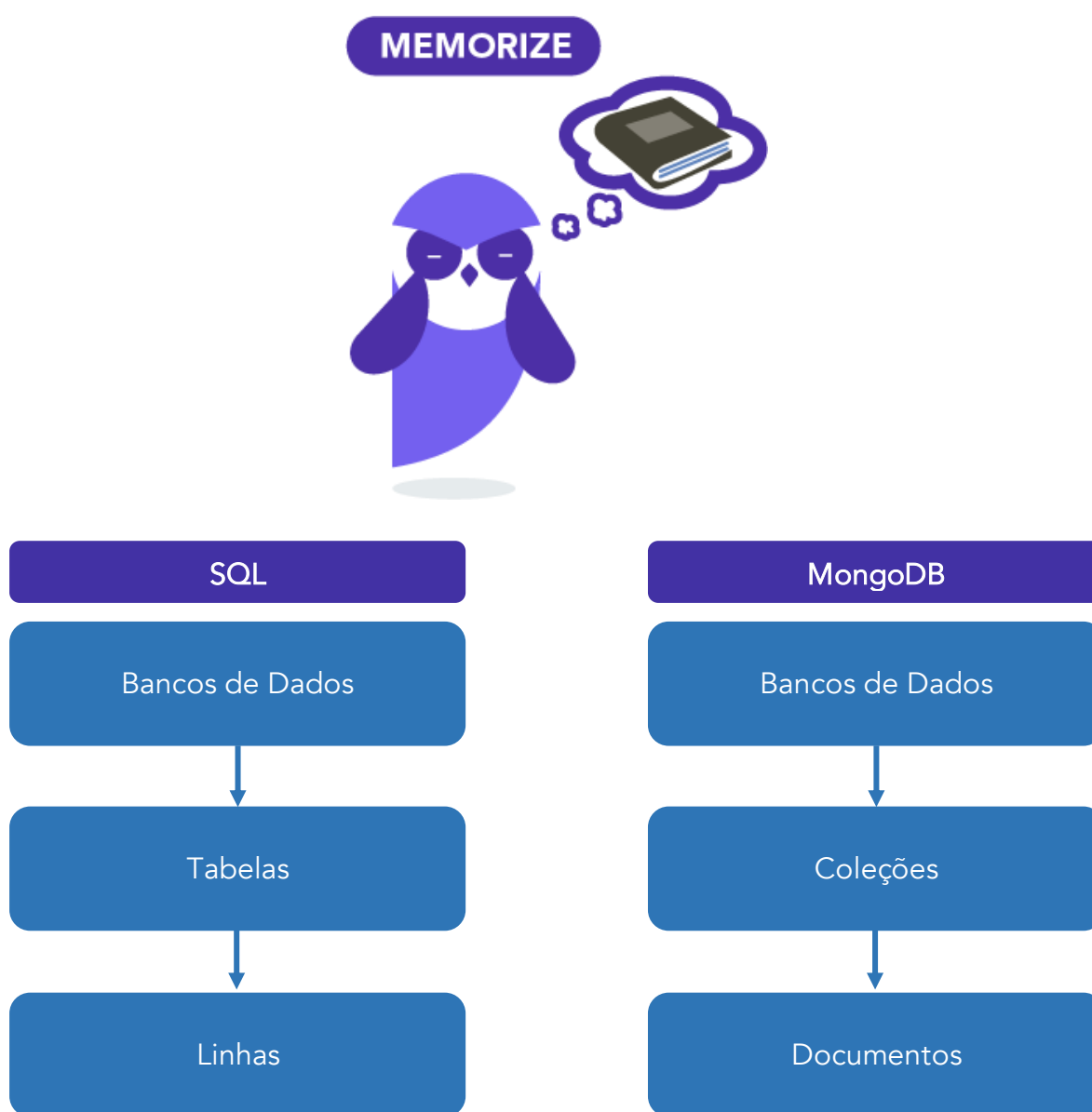
Acima, temos um arquivo JSON que incorpora alguns conjuntos de campos e valores. Os campos representados dentro de um mesmo arquivo devem ser únicos, enquanto os valores podem se repetir. Todo esse conjunto informações, as 4 linhas, representam **um documento**. Esse documento seria **equivalente a uma linha em um banco de dados SQL**.

Esses documentos ficam agregados em um conjunto de documentos, chamado de **coleções** (ou *collections*). Essas coleções podem reunir documentos de um mesmo tipo, de um mesmo assunto ou coletados de uma mesma fonte. Analogamente, as **coleções** são consideradas **as tabelas do MongoDB**.



Talvez você tenha percebido meu foco em traçar paralelos entre um banco de dados tradicional, SQL, e o MongoDB - e isso não é por qualquer motivo. Essa é uma cobrança recorrente em concursos acerca do SGBD. Então preste atenção, que temos três estruturas aqui que você precisa saber.

- **Bancos de Dados** = conjunto de coleções – equivale a um banco de dados, na modelagem relacional
- **Coleções** = conjunto de documentos – equivale a uma tabela, na modelagem relacional
- **Documentos** = arquivos JSON, XML e BSON – equivale às tuplas (linhas/row), na modelagem relacional



Veja a cobrança recente desse assunto.

(FGV/SEFAZ MT/2023) O Mongo DB é citado frequentemente como um dos bancos de dados não relacionais (NoSQL) mais utilizados no mundo.

Assinale a opção que representa o elemento disponível nesse sistema que mais se assemelha à ideia de tabela relacional.

- a) Cluster.
- b) Collection
- c) Database.
- d) Document.
- e) Index.

Comentários:

Conforme vimos, o elemento análogo às tabelas relacionais, no MongoDB, são as *collections*.
(Gabarito: Letra B)

Então, apenas a título de exemplo, se convertêssemos esse documento abaixo, que está no formato de arquivos JSON, para uma tabela, teríamos a seguinte estruturação:

EXEMPLIFICANDO



```
{  
  "nome": "Felipe",  
  "idade": 30,  
  "cargo": "Professor",  
  "telefone" : ["4444-8888", "1234-4321"]  
}
```



nome	idade	cargo	telefone
------	-------	-------	----------

Felipe	30	Professor	4444-8888, 134-4321
--------	----	-----------	---------------------

Sintaxe

A sintaxe do MongoDB é própria – aqui não usaremos SQL, usaremos os comandos próprios do SGBD (justamente por se tratar de um **NoSQL**, né haha).

Os comandos no MongoDB seguem o mesmo padrão de algumas linguagens de programação – é necessário um par de parênteses () para executar o comando - pois estamos executando uma função, que pode receber determinados parâmetros.

Por exemplo:

```
db.collection.count()
```

O comando acima está realizando a função *count* (contagem) na coleção, dentro do banco de dados. Vamos listar os principais comandos, e suas sintaxes.

Inserir documento na coleção:

```
db.collection.insertOne({})
```

```
db.collection.insertMany({})
```

O comando acima irá **inserir um documento** (insertOne), **ou vários documentos** (insertMany) na nossa coleção. Dentro das chaves, especificamos os pares campo:valor. Digamos que vamos inserir meu nome, e minha idade, num banco de dados:

```
db.collection.insertOne({"nome": "Felipe Mathias", "idade": 30})
```

Consultar documentos

```
db.collection.find({})
```

Com essa sintaxe, iremos procurar algum termo específico na nossa coleção de documentos. Se você quiser encontrar documentos que contenham a idade = 30:

```
db.collection.find({"idade": 30})
```

Sobre essa consulta, podemos fazer uma **projeção**. Isso permitirá encontrarmos campos específicos de um documento. Podemos fazer isso através de uma operação binária, onde o número 1 habilitará o campo, e 0 desabilitará. Digamos que, na consulta de pessoas com idade = 30, queremos retornar também o nome, mas não o endereço:

```
db.collection.find({"idade": 30}, {"nome": 1, "endereço": 0} )
```

Atualizar documentos

```
db.collection.updateOne({ }, {$set: {}})
```

```
db.collection.updateMany({ }, {$set: {}})
```

Podemos atualizar um (updateOne) ou vários (updateMany) documentos ao mesmo tempo. Uma coisa importante é que os comandos em MongoDB usam um padrão chamado de **camel case**, onde a palavra começa minúscula, mas as próximas palavras são maiúsculas (por exemplo, se juntássemos *camel case* em uma só palavra, ficaria *camelCase*).

Vamos atualizar o nosso arquivo. Digamos que passou 1 ano do lançamento desse curso, e queremos atualizar minha idade para 31 anos.

```
db.collection.updateOne({nome: "Felipe Mathias"},{$set:{idade: 31}})
```

Excluir documentos

```
db.collection.deleteOne({})
```

```
db.collection.deleteMany({})
```

O comando de exclusão segue o mesmo padrão, de excluirmos um ou vários. Podemos, por exemplo, excluir todos os documentos de um banco de dados de uma Fazenda pública, onde o status de devedor contumaz seja *true*

```
db.collection.deleteMany({contumaz: true })
```

Gerar documentos em modo leitura

```
db.collection.find().pretty()
```

Esse comando é usado para "embelezar" o nosso texto, fazendo com que ele fique com formatação legível, com indentações e quebras de linha. Definimos o que queremos achar no comando **find** e executamos o comando **pretty** em seguida:

```
db.collection.find({nome: "Felipe Mathias"}).pretty()
```

Existem alguns outros comandos que são "acessórios", ou seja, assim com o **pretty**, eles vêm ao fim do comando primário. São eles:

- `db.collection.find().limit()` – Usado para limitar a quantidade de resultados
- `db.collection.find().sort()` – Ordenará os resultados de acordo com algum parâmetro
- `db.collection.countDocuments()` – contará a quantidade de documentos em uma coleção
- `db.collection.findOneAndUpdate({}, {})` – Irá encontrar um registro específico e atualizá-lo

Lembrando que os valores **db** e **collection** podem ser modificados de acordo com o banco de dados e coleção, respectivamente, que estivermos trabalhando.

QUESTÃO DE PROVA



(FGV/TJ RO/2021) Com referência ao MongoDB, nas consultas usando o método `find` é possível exibir o resultado formatado por meio do método:

- a) `format`;
- b) `organize`;
- c) `pretty`;
- d) `structure`;
- e) `tidy`.

Comentários:

Como vimos, quando queremos formatar um arquivo para leitura, ou seja, queremos o deixar “bonito”, usamos o comando `pretty`. (Gabarito: Letra C)

Dump

Quando você vir o termo “*dump*” no contexto de bancos de dados, sempre tenha em mente que esse termo se refere ao **processo de backup**. No MongoDB, o **`mongodump`** é um utilitário que cria

uma exportação binária do conteúdo de um banco de dados. A sintaxe base do mongodump é a seguinte:

`mongodump [opções]`

As opções de uso mais comum são:

- **--host:** O endereço do servidor MongoDB.
- **--port:** A porta em que o servidor MongoDB está ouvindo (padrão é 27017).
- **--username:** O nome de usuário para autenticação no servidor MongoDB.
- **--password:** A senha correspondente ao nome de usuário.
- **--authenticationDatabase:** O banco de dados para autenticação.
- **--db:** O nome do banco de dados a ser copiado.
- **--out:** O diretório onde os dados serão salvos.

Para pegar esse arquivo gerado e fazer uma restauração dos dados, isso é, recriar o banco de dados a partir desse arquivo, usamos o **mongorestore**.

(FCC/MPE PE/2018) No gerenciador de bancos de dados MongoDB, os comandos para exibir a lista de bancos de dados presentes no servidor e para gerar um backup são, respectivamente,

- a) show dbs e mongodump.
- b) list dbs e mongodrop.
- c) copy dbs e mongouse.
- d) use dbs e mongoname.
- e) serve dbs e mongofind.

Comentários:

Para mostrarmos a lista de bancos de dados existentes, usamos o comando *showdbs*. Já para backup, usamos o utilitário *mongodump*. (Gabarito: Letra A)

Redis

Redis é um **SGBD NoSQL** voltado, primariamente, para **bancos de chave-valor**. Ele é considerado um **armazenador de estrutura de dados in-memory**, isso é, um banco de dados que armazena os dados na memória, que usa primariamente o cache do computador.

QUESTÃO DE PROVA



(AOCF/IF MA/2023) O Redis é uma solução amplamente utilizada no contexto de armazenamento em cache. Assinale a alternativa que descreve corretamente o Redis e suas características.

- a) Redis é um banco de dados relacional que armazena dados em tabelas e oferece cache para consultas SQL.
- b) Redis é uma ferramenta de indexação e busca de texto completo que pode ser usada para armazenar e recuperar documentos.
- c) Redis é um sistema de gerenciamento de conteúdo que utiliza o armazenamento em cache para melhorar o desempenho das páginas.
- d) Redis é um mecanismo de armazenamento em cache que utiliza estruturas de dados em memória, como strings, listas e conjuntos, para melhorar a performance.
- e) Redis é um servidor de aplicação que utiliza o armazenamento em cache para melhorar a performance de aplicações web.

Comentários:

Vamos analisar as alternativas.

- a) Errado. O Redis não armazena dados em tabelas.
- b) Errado. Embora forneça essa ferramenta, o Redis não se resume a isso.
- c) Errado. O Redis é um sistema gerenciar de banco de dados, não de conteúdo.
- d) Certo. O Redis armazena em memória cache conteúdos diversos, como strings, imagens, e associa a esses conteúdos uma chave, formando o par chave-valor.
- e) Errado. Novamente, o Redis é um sistema gerenciar de bancos de dados.

A alternativa correta é, portanto, a letra D. (Gabarito: Letra D)

O Redis oferece uma interface de usuários para a realização de comandos, chamada de **Redis Command Line Interface (Redis-CLI)**, um programa de terminal (assim como é o seu prompt de comando, no Windows). Os usuários usam principalmente o modo REPL (Read Eval Print Loop) – um modo interativo, onde são escritos os comandos Redis e os usuários recebem as respostas.

A visualização e otimização dos dados no Redis pode ser feito a partir da ferramenta **RedisInsight**, fornecendo uma plataforma interativa e em tempo real, oferecendo uma plataforma mais “amigável” (através de uma Interface Gráfica de Usuário – GUI).

A **inserção de dados** no Redis é feita a partir de uma **stream (a Redis Stream)**, que faz uma inserção definitiva dos dados em disco – isso é, esses dados não podem ser alterados depois da inserção. A partir dessa inserção, uma cópia dos dados é levada à memória dos usuários, que trabalham a partir dela. Dessa forma, é possível que usuários manipulem os dados, e a fonte original de dados permaneça intacta, servindo como fonte única de “verdade”, além de servir como um log dos dados.

Usualmente as streams existem dois agentes: o produtor, e o consumidor - funcionando num sistema chamado de *pub/sub* (ou *publisher/subscriber*).

- **Produtor:** responsável por adicionar novos eventos ao Stream – é o aplicativo que insere os dados para serem enviados ao Redis Stream
- **Consumidor:** é a entidade que consome os dados, lendo os eventos na ordem que são adicionados

Como falamos, os dados do Redis são do par chave-valor, mas eles podem ser armazenados de diversas formas:

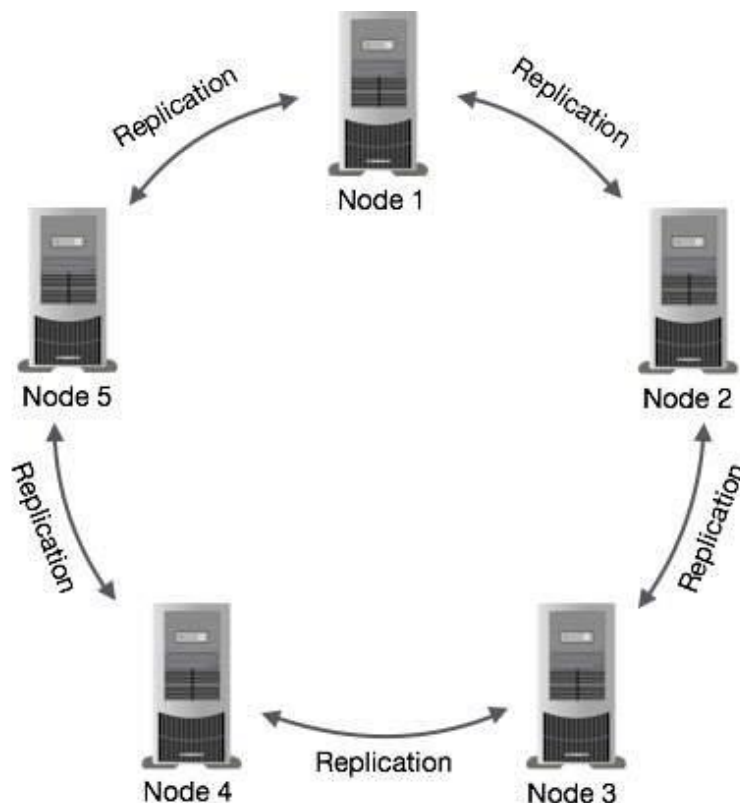
- String – são os dados mais simples, correspondendo a um simples par chave e valor.
- Set (conjuntos) – são coleções não ordenadas de strings.
- Listas – coleções ordenadas de strings. É possível que você adicione elementos ao início (esquerda) ou ao fim (direita) de uma lista.
- Hashes – estruturas de dados que permite armazenar conjuntos de chave-valor em apenas uma única chave, semelhante a um dicionário, mapa ou índice.
- Sorted sets (conjuntos ordenados) – semelhante aos sets, mas cada set no conjunto terá uma pontuação associada a ele, usada para classificar o conjunto em ordem crescente ou decrescente
- Bitmaps – sequências de bits compactadas, servindo para operações lógicas (AND, OR, NOT e XOR)
- HyperLogLogs – estruturas probabilísticas usadas para estimar a cardinalidade em um conjunto grande de dados

Cassandra

Apache Cassandra é um **NoSQL** destinado especialmente para lidar com sistemas de computação distribuída (usualmente associado ao ecossistema do HADOOP), desenvolvido pela Apache, **orientado a colunas** (ou **wide column**).



A sua principal característica é operar em sistemas de computação distribuída – isso é, trabalha com vários servidores (computadores) diferentes, que funcionam como **nós**, que são réplicas entre si (literalmente, cópias), que sofrem sincronização de forma síncrona ou assíncrona.



Os componentes do Cassandra incluem:

- **Nós (nodes)** – é onde os dados são armazenados
- **Data center** – uma coleção de nós relacionados
- **Cluster** – componente que contém um ou mais data center
- **Filtro bloom** – algoritmos de teste de integridade
- **Commit log** – mecanismo de recuperação para quedas do sistema
- **CQL(cqlsh)** – Cassandra Query Language – utilitário nativo que permite executar comandos e consultas

Vamos entender a organização dos objetos do Cassandra. Começamos com a unidade mais básica de informação: as **colunas**. Cada coluna é como um par de chave e valor, em que o nome da coluna seria equivalente à chave, e as linhas preenchidas nessa coluna seriam seus respectivos valores. Uma coluna é **composta de**, ao mínimo, **três valores**:

Chave/Nome da Coluna
Valor
Carimbo de tempo (<i>timestamp</i>)

A chave ou nome da coluna representa o tipo de valor que estamos representando; o valor é o dado atribuído à chave; o timestamp é um número que representa a quantidade de milissegundos percorridos desde o “Epoch Unix” (1 de janeiro de 1970 00:00:00). Então, por exemplo, uma coluna pode ser preenchida da seguinte forma:

User
anon
1618444368000

É possível também termos o que é chamado de **supercolunas**, que nada mais são que uma coluna com outras colunas agregadas a ela. Elas parecem-se com isso:

User	
User	Psswd
anon	1234
1618444368000	1618444368001

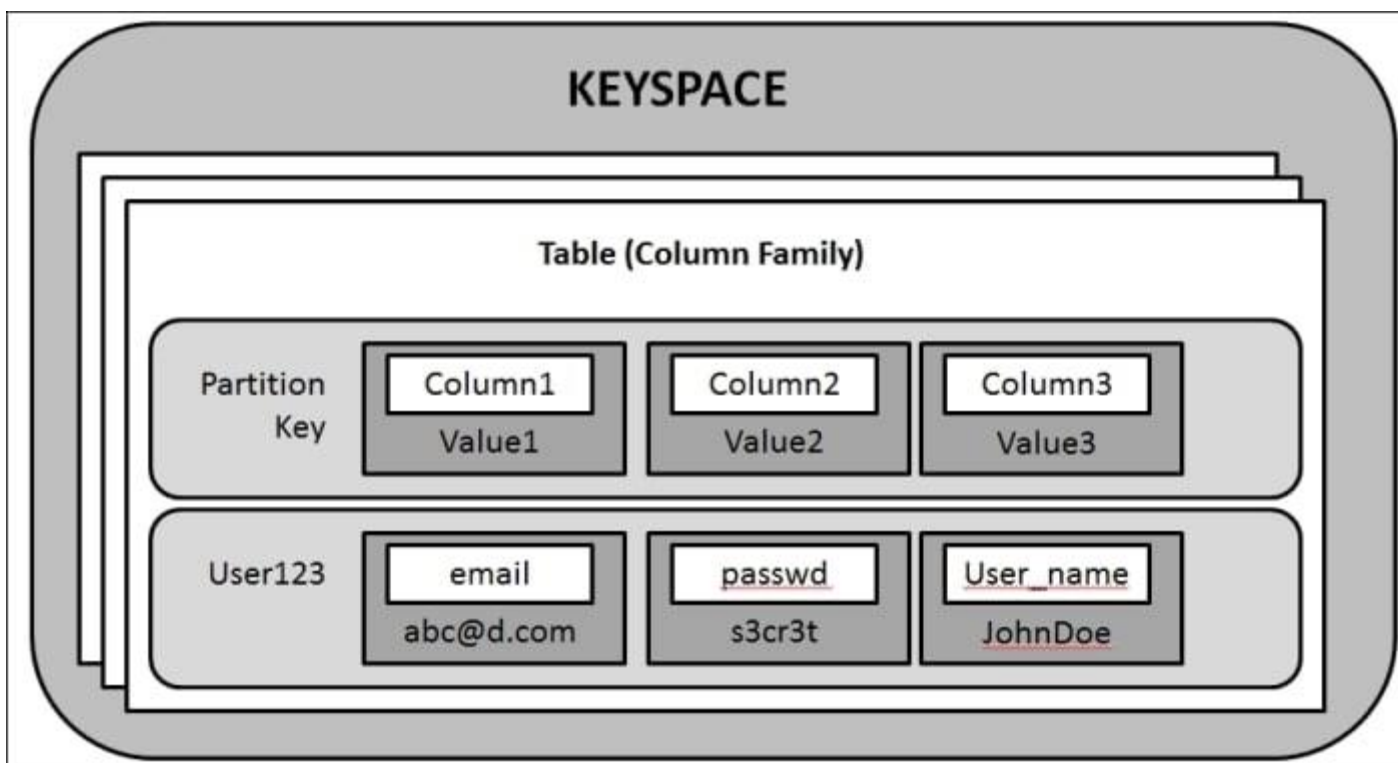
Uma agregação de diferentes colunas, que são tratadas isoladamente, são chamadas de **linhas** (ou *rows*). Cada linha corresponde a um conjunto de dados, identificados por uma chave (*row key*) usualmente tratando de um mesmo objeto. Continuando esse exemplo, podemos ter uma linha de dados acerca do *login* de um usuário.



Agora, uma **tabela** em um banco de dados Cassandra, chamada também de **Column Family (Família de Colunas)** é nossa penúltima forma de agregação de dados, e será composta por diversas linhas - de formas a termos uma estrutura similar a uma estrutura tabular, lembrando que cada linha tem diversas colunas que podem diferir, então resalto o "similar".

Dentro dessas famílias de coluna, é possível criar um conjunto de linhas, que receberá o nome de **partição**, e funcionará como uma "linha de linhas".

Por fim, envolvendo todos os conjuntos de objetos no Cassandra temos o **Keyspace**. Ele funciona quase como um banco de dados, no modelo relacional, envolvendo tudo que temos armazenado, além de fornecer um esquema de estruturação. Veja:



(AOC/MJSP/2020) O Cassandra DB é um banco NoSQL baseado em um esquema flexível. Sabendo disso, assinale a alternativa que apresenta o nome do elemento do Cassandra DB que seja equivalente ao 'esquema' de um banco de dados relacional.

- a) Family.
- b) Widespace.
- c) Keyspace.
- d) Collection.
- e) Tablespace.

Comentários:

A estrutura que equivale ao esquema dos SGBDr é o Keyspace. (Gabarito: Letra C)

QUESTÃO DE PROVA



Assim como no MongoDB, é comum relacionarmos as estruturas do Cassandra com as estruturas tradicionais dos bancos de dados relacionais. Então, anote!



(Inédita/Prof. Felipe Mathias) Assinale a alternativa que corresponde ao conceito de tabelas do modelo relacional, no contexto do Apache Cassandra:

- a) Keyspace
- b) Tabelspace
- c) Column
- d) Column Family
- e) Row

Comentários:

A estrutura semelhante com as tabelas do modelo relacional, no Apache Cassandra, chama-se Column Family. (Gabarito: Letra D)

QUESTÃO DE PROVA



(FEPESE/CELESC/2019) Analise as afirmativas abaixo sobre o Apache Cassandra.

1. Possui uma arquitetura descentralizada, de modo que todo nó no cluster é idêntico.
2. É um sistema gerenciador de bancos de dados do tipo NoSQL.
3. A replicação das atualizações ocorre necessariamente de forma síncrona por todo o cluster.

Assinale a alternativa que indica todas as afirmativas corretas.

- a) É correta apenas a afirmativa 1.
- b) São corretas apenas as afirmativas 1 e 2.
- c) São corretas apenas as afirmativas 1 e 3.
- d) São corretas apenas as afirmativas 2 e 3.

e) São corretas as afirmativas 1, 2 e 3.

Comentários:

Vamos analisar cada uma das afirmativas.

1. Certo. Cada nó do Apache Cassandra é uma réplica dos demais, isso é, uma cópia idêntica, criando uma malha de nós formando um cluster de réplicas.

2. Certo. Cassandra, de fato, é um SGBD NoSQL, destinado primariamente a dados do tipo colunares, ou *wide-column*.

Errado. A replicação pode ser tanto síncrona, quanto assíncrona.

Portanto, corretas as alternativas 1 e 2. (*Gabarito: Letra B*)

Demais SGBDs

Aqui reunirei os demais SGBDs de NoSQL com suas características mais importantes. Será uma abordagem rápida, já que são assuntos que não costumam cair. São dois os tipos de informação que você deve extrair dessa lista:

- O tipo de NoSQL que o SGBD é responsável por lidar
- Quaisquer componentes de destaque que merecem atenção (irei ressaltá-los)

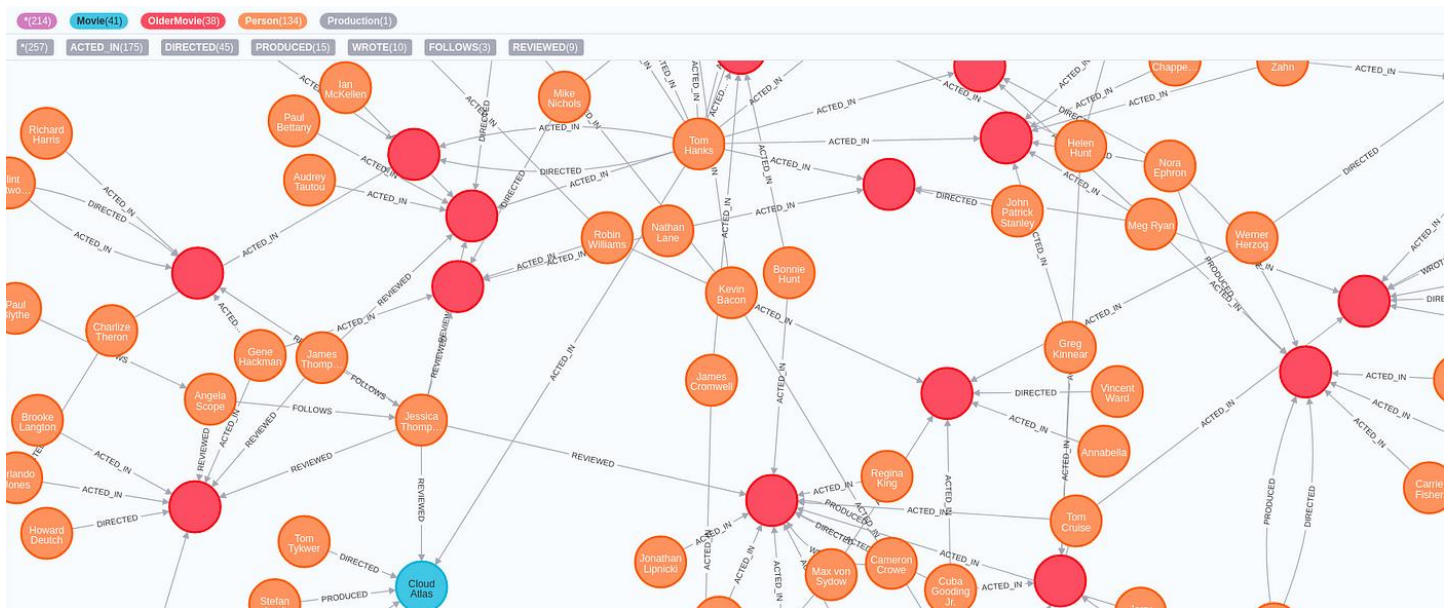
Com isso em mente, vamos lá!

Neo4j

O Neo4j é um banco de dados de grafos NoSQL, projetado especificamente para armazenar e consultar dados altamente conectados. Ele difere dos bancos de dados relacionais e de outros bancos de dados NoSQL em sua estrutura de armazenamento e consulta, que são otimizadas para lidar com dados que têm muitas relações complexas.

No Neo4j, os dados são modelados como grafos, que consistem em nós, relacionamentos e propriedades. Os nós representam entidades individuais, como pessoas, produtos ou lugares, e os relacionamentos representam as conexões entre essas entidades. Cada nó e relacionamento podem ter propriedades associadas que descrevem características específicas.

Uma das características distintivas do Neo4j é sua capacidade de realizar consultas complexas em grafos de forma eficiente. Ele utiliza uma linguagem de consulta chamada Cypher, que é semelhante a uma linguagem de consulta de grafo, permitindo que os usuários expressem padrões de conexão entre os nós e relacionamentos. Isso torna o Neo4j adequado para uma ampla gama de casos de uso, incluindo redes sociais, recomendações, análise de redes e detecção de fraudes.



DynamoDB

O **Amazon DynamoDB** é um serviço de **banco de dados NoSQL totalmente gerenciado** fornecido pela Amazon Web Services (AWS). Ele foi projetado para oferecer desempenho rápido e escalabilidade automática, lidando com dados no formato **chave-valor**, o que o torna uma escolha popular para aplicativos que exigem acesso rápido a grandes quantidades de dados com requisitos de escalabilidade dinâmica.

O DynamoDB é construído em torno de alguns componentes principais que contribuem para sua eficiência e flexibilidade. Um desses componentes é a **tabela**, que é a **unidade básica de armazenamento de dados** no DynamoDB. As tabelas são compostas por itens e cada item é uma coleção de atributos, onde os dados são armazenados. As tabelas no DynamoDB são esquemas flexíveis, o que significa que cada item em uma tabela não precisa ter o mesmo conjunto de atributos.

CouchDB

O **CouchDB** é um **banco de dados de documentos NoSQL**, desenvolvido pela Apache Software Foundation. Ele é projetado para armazenar, acessar e gerenciar **dados semi-estruturados**, como documentos JSON, de uma forma distribuída e tolerante a falhas.

No CouchDB, os dados são armazenados como documentos individuais, que são estruturados no formato JSON. Cada documento contém um conjunto de pares de chave-valor, onde os valores podem ser strings, números, booleanos, arrays ou objetos JSON aninhados. Esses documentos são organizados em coleções chamadas de "bancos de dados".

Uma das características distintivas do CouchDB é sua **arquitetura distribuída** e tolerante a falhas. Ele utiliza um modelo de **replicação bidirecional**, onde os **dados podem ser sincronizados entre**

múltiplos nós de banco de dados de forma automática e transparente. Isso significa que os dados podem ser acessados e modificados em qualquer nó do cluster, e as atualizações são replicadas entre os nós de forma consistente.

HBase

O **Apache HBase é um banco de dados distribuído e orientado a colunas**, desenvolvido como parte do ecossistema Apache Hadoop. Ele foi projetado para fornecer armazenamento e acesso eficiente a grandes volumes de dados, especialmente em ambientes distribuídos e escaláveis.

No HBase, **os dados são organizados em tabelas**, assim como em um banco de dados relacional, mas com uma estrutura de armazenamento diferente. Em vez de armazenar dados em linhas contíguas, como em um banco de dados relacional, **o HBase armazena dados em colunas contíguas**, o que permite uma recuperação mais eficiente dos dados.

A estrutura básica de um banco de dados HBase é composta por tabelas, famílias de colunas e células. As tabelas contêm linhas de dados, identificadas por uma chave primária única, e são divididas horizontalmente em regiões que são distribuídas entre os nós do cluster. Cada linha de uma tabela pode conter uma ou mais famílias de colunas, que por sua vez contêm uma ou mais colunas. As células de dados são armazenadas nas interseções das linhas e colunas.

Elasticsearch

O **Elasticsearch é um sistema de busca e análise distribuído** e altamente escalável, muitas vezes classificado como um motor de busca NoSQL. Embora **o Elasticsearch não seja tradicionalmente categorizado como um sistema de gerenciamento de banco de dados (SGBD) NoSQL, ele desempenha um papel fundamental no ecossistema NoSQL**, especialmente para casos de uso relacionados à indexação e pesquisa de dados não estruturados.

Embora o Elasticsearch não seja um banco de dados no sentido tradicional, ele compartilha algumas semelhanças com os sistemas de banco de dados NoSQL. Por exemplo, ele armazena dados em formato JSON e oferece uma API RESTful para consulta e manipulação dos dados. Além disso, o Elasticsearch é altamente escalável e distribuído, permitindo o processamento de grandes volumes de dados em clusters de vários nós.

No entanto, o principal foco do Elasticsearch está na **indexação e pesquisa de dados**, especialmente para casos de uso como busca textual, análise de logs, monitoramento de métricas e análise de dados em tempo real. Embora possa ser usado para armazenar e recuperar dados, não é projetado para substituir completamente um banco de dados NoSQL tradicional em todos os casos de uso.

QUESTÃO DE PROVA



(AOC/PRODEB/2018) Com base nos sistemas de banco de dados NoSQL, assinale a alternativa que correlaciona corretamente os SGBD's no NoSQL e seus modelos estruturais.

- a) MongoDB: Modelo Orientado a Colunas – Cassandra: Modelo Baseado em Grafos – Neo4J: Modelo Orientado a Documentos – Redis: Modelo Chave-Valor.
- b) Cassandra: Modelo Orientado a Colunas – Neo4J: Modelo Baseado em Grafos – Redis: Modelo Orientado a Documentos – MongoDB: Modelo Chave-Valor.
- c) Redis: Modelo Orientado a Colunas – Cassandra: Modelo Baseado em Grafos – MongoDB: Modelo Orientado a Documentos – Neo4J: Modelo Chave-Valor.
- d) Neo4J: Modelo Orientado a Colunas – Cassandra: Modelo Baseado em Grafos – MongoDB: Modelo Orientado a Documentos – Redis: Modelo Chave-Valor.
- e) Cassandra: Modelo Orientado a Colunas – Neo4J: Modelo Baseado em Grafos – MongoDB: Modelo Orientado a Documentos – Redis: Modelo Chave-Valor.

Comentários:

A única afirmativa que faz uma associação correta entre o SGBD e o tipo de dado que ele suporta é a letra E. *(Gabarito: Letra E)*

QUESTÕES COMENTADAS

01. (FUNDATEC/PROCERGS/2023) O modelo de banco de dados relacional representa o banco de dados como uma coleção de relações. Informalmente, cada relação se assemelha a uma tabela de valores ou, até certo ponto, a um arquivo simples de registros. É chamado de arquivo simples porque cada registro tem uma estrutura simples linear ou plana. Qual banco abaixo é um banco de dados relacional?

- a) MongoDB.
- b) CouchDB.
- c) Redis.
- d) Oracle.
- e) NoSQL.

Comentários:

Trouxe essa questão para que você possa perceber a importância do conhecimento agregado em TI. Você não sabe a resposta - mas sabe quais não são.

As alternativas A, B e C apontam SGBDs NoSQL. A letra E é o gênero de bancos de dados que não envolvem o uso de SQL. Por fim, resta a letra D - o Oracle. Oracle é um banco de dados relacional mantido pela Amazon, que usa uma linguagem especial adaptada do SQL - o PL SQL.

Gabarito: Letra D

02. (FEPESE/PREF. B. CAMBORIÚ/2023) Analise as afirmativas abaixo sobre MongoDB.

1. Um documento é a unidade básica de dados e é mais ou menos equivalente a um registro; é um banco de dados relacional, porém mais expressivo.
2. Uma coleção contém similaridades com e remete a uma tabela no modelo relacional.
3. Cada instância do MongoDB pode conter um único banco de dados que por sua vez pode conter múltiplas coleções.

Assinale a alternativa que indica todas as afirmativas corretas.

- a) É correta apenas a afirmativa 3.
- b) São corretas apenas as afirmativas 1 e 2.
- c) São corretas apenas as afirmativas 1 e 3.
- d) São corretas apenas as afirmativas 2 e 3.
- e) São corretas as afirmativas 1, 2 e 3.

Comentários:

Vamos analisar os itens.

1. Certo. O documento, unidade básica de dado do MongoDB, equivale, aproximadamente, a um registro (linha) de uma tabela de um SGBDr.
2. Certo. As coleções, dadas as devidas margens, se assemelham a tabelas dos modelos relacionais.
3. Errado. Cada instância pode conter múltiplos bancos de dados, que, por si, podem conter múltiplas coleções.

Corretas, portanto, as afirmativas 1 e 2.

Gabarito: Letra B

03. (FUNDATEC/IFC/2023) Com o advento de tipos diferentes de sistemas de gerenciamento de bancos de dados (SGBDs), o começo do desenvolvimento de qualquer software envolve a escolha de SGBD para armazenar dados. Qual SGBD é orientado a documentos em que o modelo de banco de dados armazena as informações como documentos e que são representados internamente como grupos de objetos que possuem chaves distintas?

- a) MongoDB.
- b) PostgreSQL.
- c) Flask SQLAlchemy.
- d) MySQL.
- e) NoSQL.

Comentários:

Das alternativas apresentadas, a única que apresenta um SGBD orientado a documentos é a letra A - mongoDB. Quanto às demais:

- PostgreSQL: SGBD relacional, que aceita abordagem orientada a objetos e realiza um mapeamento objeto-relacional;
- Flask SQLAlchemy: extensão do Flask, framework web da linguagem Python;
- MySQL: SGBD relacional;

Gabarito: Letra A

04. (AOC/MJSP/2020) Em um banco de dados MongoDB, uma projeção apresenta quais campos, em um documento, são incluídos ou excluídos da saída final. Um administrador de banco de dados MongoDB do MJSP necessita realizar uma projeção. Sabendo que tal projeção assume a forma de

uma expressão JSON que consiste em pares, assinale a alternativa que apresenta corretamente esses pares.

- a) key:value.
- b) key:document.
- c) value:document.
- d) primarykey:document.
- e) primarykey:foreingkey.

Comentários:

Os arquivos JSON são arquivos em texto responsáveis por carregar dados entre sistemas. Sua estrutura é feita em pares de chaves e valores, ou *key:value*.

Gabarito: Letra A

05. (FGV/CGU/2022) Sobre o MongoDB v5.0, considere as afirmativas a seguir.

- I. Todos os documentos de uma coleção (collection) devem possuir o mesmo número de campos (fields).
- II. Um campo presente em diferentes documentos de uma collection pode ter diferentes tipos de dados ao longo dos documentos.
- III. MongoDB oferece a capacidade de validar esquemas durante operações de inserção (insertion) e atualização (update).

Está correto o que se afirma em:

- a) somente I;
- b) somente III;
- c) somente I e II;
- d) somente II e III;
- e) I, II e III

Comentários:

Vamos analisar cada uma das afirmativas:

- I. Errado. Podemos ter vários campos diferentes em cada documento da coleção, não há essa restrição.
- II. Certo! Não há uma restrição de dados, como nos bancos relacionais, por exemplo, para o MongoDB.
- III. Certo. É possível a validação através de um conjunto de regras de validação.

06. (CEBRASPE/TRT 8/2022) Foi solicitada a um analista do TRT, através de chamado técnico, uma análise de desempenho no MongoDB, com o objetivo de investigar e verificar se o banco de dados estaria operando com desempenho aquém do esperado e de forma degradada.

Considerando essa situação hipotética, assinale a opção que indica o componente do MongoDB, a ser investigado pelo analista, que permita a coleta de informações detalhadas sobre as operações executadas em certa instância MongoDB.

- a) Database Profiling
- b) MongoDB Aggregation Pipeline
- c) Shards
- d) Mongodump
- e) MongoDB Compass

Comentários:

Vamos comentar o que é cada uma das afirmativas:

- a) Certo. Database Profiling: é o nosso gabarito. O Database Profiler é uma ferramenta no MongoDB que permite capturar informações detalhadas sobre as operações executadas em um banco de dados. Ele registra dados como consultas, atualizações, inserções e exclusões, bem como o desempenho dessas operações, como tempo de execução e uso de recursos.
- b) Errado. Aggregation Pipeline: é uma ferramenta usada para fazer o processo de transformar e analisar dados. Ele combina, filtra, projeta, agrupa, ordena e realiza cálculos em documentos de uma coleção.
- c) Errado. Shards: são servidores individuais, onde cada shard armazena uma parte dos dados de um cluster distribuído.
- d) Errado. Mongodump: é um comando que permite criar backups dos dados de um servidor. Ele cria um arquivo de despejo (dump) dos dados, índices e metadados. Esses arquivos podem ser usados posteriormente com o 'mongorestore', para restaurar os dados.
- e) Errado. Compass: é uma interface gráfica de usuário (GUI) que permite visualizar, explorar, analisar e interagir com os dados armazenados no MongoDB.

Portanto, correta a letra A.

07. (CEBRASPE/TRT 8/2022) O Mongodump é um utilitário para backup no MongoDB 6.0 que

- a) cria uma exportação binária do conteúdo de um banco de dados incluindo os Sharded clusters.
- b) exporta os dados em um arquivo contendo comandos de criação de estrutura e inserção de dados em formato JSON.
- c) exporta os dados em formato nativo do MongoDB contendo todos os comandos necessários para criar os bancos de dados, com exceção dos replica sets, que usam outra solução de backup.
- d) exporta os dados em formato BSON com os comandos de criação e de manipulação de dados, incluindo os Standalone deployments.
- e) exporta os dados em formato nativo do MongoDB contendo todos os comandos necessários para criar os bancos de dados, com exceção dos sharded clusters, que usam outra solução de backup.

Comentários:

Como vimos na afirmativa anterior, o Mongodump cria arquivos que podem servir de backup para recuperações do banco de dados. Ele exporta arquivos em formatos binários (no formato BSON, para ser mais preciso), contendo dados das transações, dos documentos e dos shards (nossos "nós").

Gabarito: Letra A

08. (CEBRASPE/SERPRO/2021) A respeito de conceitos de NoSQL orientado a grafos, orientado a documentos e orientado a colunas, julgue o item a seguir.

Cypher query language, linguagem de consulta de gráfico do Neo4j que possibilita aos usuários o armazenamento e a recuperação de dados do banco de dados de gráficos, permite declarar o que se deseja selecionar, inserir, atualizar ou excluir desses dados de gráficos.

Comentários:

Cypher é a linguagem que utilizamos no Neo4j, um operador de banco de dados NoSQL orientado a grafos. Ele permite justamente isso – selecionar, inserir, atualizar e excluir dados nos nossos grafos (aqui é um erro de tradução – Graph são grafos, não gráficos).

Gabarito: Certo

09. (CEBRASPE/SERPRO/2021) A respeito de conceitos de NoSQL orientado a grafos, orientado a documentos e orientado a colunas, julgue o item a seguir.

Uma coleção e um documento, no MongoDB, são equivalentes à tabela e à linha, no Modelo Relacional de Dados.

Comentários:

Perfeito! Lembrem do nosso esquema, coleções são equiparadas às tabelas do modelo relacional, enquanto documentos equiparam-se a linhas.

Gabarito: Certo

10. (IDECAN/SEFAZ RR/2022) Saulo trabalha como implementador de software e recebe a demanda de implementar um banco de dados NoSQL para que seja usado em conjunto com o site da empresa. Selecione o tipo de banco de dados que Saulo poderá usar.

- a) MariaDB
- b) Oracle SQL
- c) PostgreSQL
- d) MongoDB
- e) MicrosoftSQL

Comentários:

Questão mais tranquila, né? Das alternativas apontadas, o único SGBD NoSQL é o MongoDB. Todos os demais são SGBDs direcionados a bancos de dados relacionais.

Gabarito: Letra D

11. (IDECAN/SEFAZ RR/2022) “Os bancos de dados NoSQL não são todos iguais. Há grandes diferenças no que se refere como a forma de armazenamento e conceitos de modelagem. Basicamente se subdividem em técnicas de armazenamento diferentes.” Sobre o assunto, assinale a alternativa que indica o banco de dados baseado na estrutura de orientação a documento.

- a) Oracle NoSQL
- b) Redis
- c) GraphBase
- d) OrientedDB
- e) MongoDB

Comentários:

Falou em SGBD NoSQL orientado a documentos, associe na hora o MongoDB. Sobre os demais SGBD:

- Oracle NoSQL: chave-valor
- Redis: chave-valor
- GraphBase: invenção do criador da questão.
- OrientedDB: invenção do criador da questão. O certo seria OrientDB.

Gabarito: Letra E

12. (FEPESE/CIASC/2017) Podem-se executar comandos e consultas em um banco de dados Apache Cassandra através do utilitário nativo da distribuição:

- a) sqlplus
- b) kasser
- c) cqlsh
- d) cqlplus
- e) cqldata

Comentários:

Como vimos no capítulo direcionado a Cassandra, para realizarmos consultas no SGBD, usamos o CQLSH – Cassandra Query Language Shell

Gabarito: Letra C

13. (IDECAN/TJ PI/2022) Marque a alternativa que indica, respectivamente, o nome de dois bancos de dados NoSQL sendo o primeiro um banco de dados in-memory, muito utilizado para armazenamentos do tipo chave-valor e o segundo um banco de dados especializado em grafos, motivo pelo qual é recomendado para aplicações que precisam armazenar informações com muitos relacionamentos de interconexão.

- a) MongoDB e GraphQL
- b) Redis e MongoDB
- c) MariaDB e GraphQL
- d) No4J e DynamoDB
- e) Redis e Neo4j

Comentários:

Primeiro, queremos um banco de dados in-memory, para dados chave-valor. Estamos falando, portanto, do Redis. O outro banco é orientado a grafos – como vimos, o maior expoente é o Neo4j.

Sobre os demais SGBDs presentes:

- MongoDB: NoSQL orientado a documentos
- GraphQL: é uma linguagem de consulta para arquiteturas REST
- MariaDB: é um SGBD para bancos de dados relacionais
- DynamoDB: NoSQL chave-valor

Gabarito: Letra E

14. (AOCP/MJSP/2020) Um cientista de dados necessita apresentar a quantidade de documentos em uma coleção do MongoDB. Sabendo disso, assinale a alternativa que apresenta corretamente o operador que o cientista precisa utilizar.

- a) `db.collection.agregate()`
- b) `db.collection.count()`
- c) `db.collection.total()`
- d) `db.collection.find()`
- e) `db.collection.sum()`

Comentários:

Quando queremos contar a quantidade de documentos numa coleção, usamos o comando `count`. Os demais comandos executarão o seguinte:

- **Agregate:** Usado para executar agregações avançadas, criando pipelines e transformando dados.
- **Total:** não é um comando específico.
- **Sum:** não é um comando autônomo. Pode usado como operador, para calcular a soma de valores.
- **Find:** usado para encontrar um documento específico numa coleção.

Portanto, correta a letra B.

Gabarito: Letra B

15. (CESGRANRO/AGERIO/2023) Na implementação de sistemas de Big Data, é comum a utilização de tecnologias de Bancos de Dados NoSQL, que têm como característica principal não seguir ou ir além do Modelo Relacional e da linguagem SQL.

Duas dessas tecnologias são os Bancos de Dados orientados a documentos e orientados a colunas, exemplificados, respectivamente, pelos softwares

- a) ArangoDB e MongoDB

- b) HBase e ArangoDB
- c) HBase e MongoDB
- d) MongoDB e ArangoDB
- e) MongoDB e HBase

Comentários:

Queremos um banco de dados orientado a documentos, e um colunar. Vamos analisar as alternativas.

- a) Errado. ArangoDB: multi-modelo; MongoDB: documentos.
- b) Errado. HBase: colunar; ArangoDB: multi-modelo
- c) Errado. HBase: colunar; MongoDB: documentos
- d) Errado. MongoDB: documentos; ArangoDB: multi-modelo
- e) Certo. MongoDB: documentos; HBase: colunar

Portanto, a alternativa correta é a letra E.

Gabarito: Letra E

LISTA DE QUESTOES

01. (FUNDATEC/PROCERGS/2023) O modelo de banco de dados relacional representa o banco de dados como uma coleção de relações. Informalmente, cada relação se assemelha a uma tabela de valores ou, até certo ponto, a um arquivo simples de registros. É chamado de arquivo simples porque cada registro tem uma estrutura simples linear ou plana. Qual banco abaixo é um banco de dados relacional?

- a) MongoDB.
- b) CouchDB.
- c) Redis.
- d) Oracle.
- e) NoSQL.

02. (FEPESE/PREF. B. CAMBORIÚ/2023) Analise as afirmativas abaixo sobre MongoDB.

1. Um documento é a unidade básica de dados e é mais ou menos equivalente a um registro; é um banco de dados relacional, porém mais expressivo.
2. Uma coleção contém similaridades com e remete a uma tabela no modelo relacional.
3. Cada instância do MongoDB pode conter um único banco de dados que por sua vez pode conter múltiplas coleções.

Assinale a alternativa que indica todas as afirmativas corretas.

- a) É correta apenas a afirmativa 3.
- b) São corretas apenas as afirmativas 1 e 2.
- c) São corretas apenas as afirmativas 1 e 3.
- d) São corretas apenas as afirmativas 2 e 3.
- e) São corretas as afirmativas 1, 2 e 3.

03. (FUNDATEC/IFC/2023) Com o advento de tipos diferentes de sistemas de gerenciamento de bancos de dados (SGBDs), o começo do desenvolvimento de qualquer software envolve a escolha de SGBD para armazenar dados. Qual SGBD é orientado a documentos em que o modelo de banco de dados armazena as informações como documentos e que são representados internamente como grupos de objetos que possuem chaves distintas?

- a) MongoDB.
- b) PostgreSQL.
- c) Flask SQLAlchemy.
- d) MySQL.
- e) NoSQL.

04. (AOCP/MJSP/2020) Em um banco de dados MongoDB, uma projeção apresenta quais campos, em um documento, são incluídos ou excluídos da saída final. Um administrador de banco de dados MongoDB do MJSP necessita realizar uma projeção. Sabendo que tal projeção assume a forma de uma expressão JSON que consiste em pares, assinale a alternativa que apresenta corretamente esses pares.

- a) key:value.
- b) key:document.
- c) value:document.
- d) primarykey:document.
- e) primarykey:foreingkey.

05. (FGV/CGU/2022) Sobre o MongoDB v5.0, considere as afirmativas a seguir.

- I. Todos os documentos de uma coleção (collection) devem possuir o mesmo número de campos (fields).
- II. Um campo presente em diferentes documentos de uma collection pode ter diferentes tipos de dados ao longo dos documentos.
- III. MongoDB oferece a capacidade de validar esquemas durante operações de inserção (insertion) e atualização (update).

Está correto o que se afirma em:

- a) somente I;
- b) somente III;
- c) somente I e II;
- d) somente II e III;
- e) I, II e III

06. (CEBRASPE/TRT 8/2022) Foi solicitada a um analista do TRT, através de chamado técnico, uma análise de desempenho no MongoDB, com o objetivo de investigar e verificar se o banco de dados estaria operando com desempenho aquém do esperado e de forma degradada.

Considerando essa situação hipotética, assinale a opção que indica o componente do MongoDB, a ser investigado pelo analista, que permita a coleta de informações detalhadas sobre as operações executadas em certa instância MongoDB.

- a) Database Profiling
- b) MongoDB Aggregation Pipeline

- d) Mongodump
- e) MongoDB Compass

07. (CEBRASPE/TRT 8/2022) O Mongodump é um utilitário para backup no MongoDB 6.0 que

- a) cria uma exportação binária do conteúdo de um banco de dados incluindo os Sharded clusters.
- b) exporta os dados em um arquivo contendo comandos de criação de estrutura e inserção de dados em formato JSON.
- c) exporta os dados em formato nativo do MongoDB contendo todos os comandos necessários para criar os bancos de dados, com exceção dos replica sets, que usam outra solução de backup.
- d) exporta os dados em formato BSON com os comandos de criação e de manipulação de dados, incluindo os Standalone deployments.
- e) exporta os dados em formato nativo do MongoDB contendo todos os comandos necessários para criar os bancos de dados, com exceção dos sharded clusters, que usam outra solução de backup.

08. (CEBRASPE/SERPRO/2021) A respeito de conceitos de NoSQL orientado a grafos, orientado a documentos e orientado a colunas, julgue o item a seguir.

Cypher query language, linguagem de consulta de gráfico do Neo4j que possibilita aos usuários o armazenamento e a recuperação de dados do banco de dados de gráficos, permite declarar o que se deseja selecionar, inserir, atualizar ou excluir desses dados de gráficos.

09. (CEBRASPE/SERPRO/2021) A respeito de conceitos de NoSQL orientado a grafos, orientado a documentos e orientado a colunas, julgue o item a seguir.

Uma coleção e um documento, no MongoDB, são equivalentes à tabela e à linha, no Modelo Relacional de Dados.

10. (IDECAN/SEFAZ RR/2022) Saulo trabalha como implementador de software e recebe a demanda de implementar um banco de dados NoSQL para que seja usado em conjunto com o site da empresa. Selecione o tipo de banco de dados que Saulo poderá usar.

- a) MariaDB
- b) Oracle SQL
- c) PostgreSQL
- d) MongoDB
- e) MicrosoftSQL

11. (IDECAN/SEFAZ RR/2022) “Os bancos de dados NoSQL não são todos iguais. Há grandes diferenças no que se refere como a forma de armazenamento e conceitos de modelagem. Basicamente se subdividem em técnicas de armazenamento diferentes.” Sobre o assunto, assinale a alternativa que indica o banco de dados baseado na estrutura de orientação a documento.

- a) Oracle NoSQL
- b) Redis
- c) GraphBase
- d) OrientedDB
- e) MongoDB

12. (FEPESE/CIASC/2017) Podem-se executar comandos e consultas em um banco de dados Apache Cassandra através do utilitário nativo da distribuição:

- a) sqlplus
- b) kasser
- c) cqlsh
- d) cqlplus
- e) cqldata

13. (IDECAN/TJ PI/2022) Marque a alternativa que indica, respectivamente, o nome de dois bancos de dados NoSQL sendo o primeiro um banco de dados in-memory, muito utilizado para armazenamentos do tipo chave-valor e o segundo um banco de dados especializado em grafos, motivo pelo qual é recomendado para aplicações que precisam armazenar informações com muitos relacionamentos de interconexão.

- a) MongoDB e GraphQL
- b) Redis e MongoDB
- c) MariaDB e GraphQL
- d) No4J e DynamoDB
- e) Redis e Neo4j

14. (AOCP/MJSP/2020) Um cientista de dados necessita apresentar a quantidade de documentos em uma coleção do MongoDB. Sabendo disso, assinale a alternativa que apresenta corretamente o operador que o cientista precisa utilizar.

- a) db.collection.agregate()
- b) db.collection.count()
- c) db.collection.total()

by db.collection.findOne()
e) db.collection.sum()

... e implementações de sistemas de Big Data, e também a adoção de tecnologias de Bancos de Dados NoSQL, que têm como característica principal não seguir ou ir além do Modelo Relacional e da linguagem SQL.

Duas dessas tecnologias são os Bancos de Dados orientados a documentos e orientados a colunas, exemplificados, respectivamente, pelos softwares

- a) ArangoDB e MongoDB
- b) HBase e ArangoDB
- c) HBase e MongoDB
- d) MongoDB e ArangoDB
- e) MongoDB e HBase

GABARITO

GABARITO



1. Letra D
2. Letra B
3. Letra A
4. Letra A
5. Letra D
6. Letra A
7. Letra A
8. Certo

9. Certo
10. Letra D
11. Letra E
12. Letra C
13. Letra E
14. Letra B
15. Letra E

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.